TECHNICAL CASE STUDY

Khazana

An offline-first, end-to-end encrypted personal finance and wealth platform. No backend. No account. No telemetry. Shipped as two independent clients over one shared financial engine.

Flutter · Dart | Next.js 16 · React 19 | Drift + SQLCipher | Dexie · Web Crypto | Tauri 2

AES-256-GCM | Ed25519 | iOS · Android · macOS · Web · Desktop

Sandhosh Sivan
Design, architecture, implementation, security model and release engineering

Solo build · 22 Jun – 25 Aug 2026

sandhoshsivan00@gmail.com

Contents

What is in this document

Written to be read three ways: as an engineering case study, as source material for a portfolio page, and as interview preparation. Every figure in it was counted from the repository, not estimated.

Section 01

Executive summary

~85,000	1,392	23	5	0
LINES HAND-WRITTEN	AUTOMATED TESTS	ENCRYPTED TABLES	PLATFORMS SHIPPED	SERVERS

The thirty-second version

Khazana is a personal finance and wealth app that keeps every rupee of your data encrypted on your own device. There is no backend, no account, and no telemetry — not as a feature, but as an architectural constraint I held for the entire build. It ships as two independently implemented clients, a Flutter app for iOS, Android and macOS and a Next.js web app that also packages as a desktop app, and both compute their numbers from the same financial engine written twice and kept honest by mirrored tests.

The two-minute version

Every personal finance app I looked at asked for something I was not willing to give: bank credentials, or a copy of my complete financial life on somebody else's server. So I built the version that does not ask. The user's PIN derives a 256-bit key through PBKDF2-HMAC-SHA256 at 600,000 iterations; on mobile that key opens a SQLCipher-encrypted SQLite file, on web it encrypts every record individually with AES-256-GCM. The key is never written to disk and is purged from memory on lock. There is no recovery path, which is a deliberate zero-knowledge trade-off rather than an oversight.

On top of that vault sits a genuine financial engine, not a spending chart. It does double-entry accounting where every journal entry sums to zero, lot-level portfolio tracking with FIFO matching, XIRR by numerical root-finding, an Indian capital-gains tax engine whose rules are versioned by disposal date, EMI amortisation with avalanche and snowball payoff simulation, insurance coverage-gap analysis, and a weighted financial-health score. All of it is pure functions with no I/O and no framework imports, which is what made it portable from Dart to TypeScript in the first place.

The part I would most want to talk through in an interview is not any one algorithm — it is the problem of shipping the same product twice without the two copies drifting apart. The answer was to make disagreement a test failure: byte-identical JSON fixtures loaded by both the Dart and the TypeScript suites, a shared specification file that drives both clients' feature gates, and cross-language assertions that pin the same inputs to the same outputs on both sides.

> **STATUS, STATED PLAINLY**
>
> Khazana is complete enough to use daily and is being prepared for the App Store, Play Store and direct desktop sale. It has **no production users yet** — the critical path is Google Play's twelve-tester, fourteen-day closed test for individual publishers, not code. The repository carries a candid twelve-item defect log, and this document reproduces the open entries rather than hiding them.

Section 02

Problem and motivation

I built Khazana because I wanted to track my own money properly and could not find a tool whose price I was willing to pay. The price was rarely money.

What the alternatives cost

APPROACH	WHAT IT ASKS FOR	WHY I REJECTED IT
Aggregator apps	Net-banking credentials or account-aggregator consent	Handing a third party read access to every account, permanently, to avoid typing. The blast radius of their breach becomes mine.
Cloud-sync trackers	A full copy of the ledger on their servers	My complete financial life, legible to their staff, their subpoenas and their next acquirer.
Spreadsheets	Nothing — but they give nothing back	No lot tracking, no XIRR, no tax view, no reminders. Every derived number is hand-maintained and therefore wrong within a month.
Wealth platforms	Assets under management	The tool's advice and my interest are structurally misaligned.

The constraint I set, and kept

One rule shaped every subsequent decision: **no server, ever**. Not a lightweight one, not "just for sync", not "just to validate licences". Once a server exists, so does an account; so does a database of other people's finances; so does a breach I have to disclose. Refusing it forced better engineering everywhere else — deterministic merge instead of a sync service, offline signature verification instead of a licensing endpoint, and a threat model I can state in one sentence.

Every architectural decision in this project is downstream of one refusal: there is no server, so there is nothing to breach, nothing to subpoena, and nothing to shut down when I stop paying for it.

What that constraint bought

- **A threat model small enough to reason about.** No JWT, OAuth, RBAC, CORS or CSRF surface exists to get wrong, because no endpoint exists. The entire security question reduces to "is on-device data unreadable without the PIN?"
- **Zero running cost.** The app has no marginal cost per user, which is what makes a one-time price honest instead of a subscription in disguise.
- **Genuine offline capability.** Not a cached degraded mode — the device holds the truth, so a flight or a dead connection changes nothing.
- **Longevity.** If I stop maintaining it tomorrow, every installed copy keeps working. Nothing expires when a server is switched off.

Why two clients rather than one

Flutter reaches iOS, Android and macOS from one codebase, but Flutter-on-web produces a canvas application that is heavy to load and awkward to make properly accessible. A separate Next.js app renders real DOM, exports as fully static files, installs as a PWA and wraps in Tauri for Windows, macOS and Linux desktop builds. Rather than treat the second client as a compromise, I treated it as a design constraint that forced the financial logic out of the UI layer entirely — which is the single decision this codebase most benefits from.

Section 03

What it does

Twenty-seven screens in the Flutter client and thirty-one routes in the web client, grouped into five areas. Features marked PRO are behind the paid unlock; everything else is free forever.

Money — the ledger

FEATURE	WHAT IT DOES
Transactions	Income, expense and investment entries with categories, merchants, notes and receipt attachments. Every entry writes a balanced pair of double-entry postings behind the scenes.
Accounts	A real chart of accounts with balances and transfers. Transfers debit the destination and credit the source, so they cancel out of net worth instead of being counted twice.
Quick add	Natural-language entry — "spent 450 on groceries at bigbasket yesterday" parses to amount, category, merchant and date.
Search	Full-text search over the whole ledger via an SQLite FTS5 virtual table with a porter tokenizer, benchmarked at roughly 0.13 ms across 10,000 transactions.
Calendar ledger	A month grid with per-day income, expense and net, plus budget indicators.
Budgets	Per-category limits with a configurable alert threshold and single-month rollover. Spend is always recomputed, never stored, so it cannot drift out of sync with the ledger.
Recurring	Rules that materialise missed occurrences on unlock, bounded so a long-stale rule cannot flood the ledger.
Goals	Targets with contributions, progress rings and a required-monthly-contribution calculation.

Invest — the portfolio

FEATURE	WHAT IT DOES
Holdings and lots	Lot-level positions across fourteen asset types — equity and ETFs, equity and debt mutual funds, gold, bonds, cash, real estate, crypto, fixed deposits, PPF/EPF, NPS, Sukanya Samriddhi, sovereign gold bonds and ULIPs.
Allocation	Donut and sunburst breakdowns by asset group, sector, industry, market cap and currency.
Sector P&L PRO	Realised and unrealised profit and loss rolled up along any dimension, computed from FIFO lot matching.
XIRR PRO	Money-weighted return across the whole portfolio or a single instrument.
Analytics PRO	Concentration risk, return distribution histogram and drill-down.
Dividends PRO	Dividend, bonus, buyback and interest events, detected from statement narrations and matched only against instruments actually held.
Tax centre PRO	Short- and long-term capital-gains estimation under Indian rules, versioned by disposal date, with the ₹1.25 lakh annual exemption pooled and allocated deterministically.
Markets, watchlist, news PRO	Opt-in index levels, a market clock, a watchlist and a news view. Market data fetches carry no personal data.

Protect — the downside

FEATURE	WHAT IT DOES
Liabilities	Loans and credit cards with EMI amortisation, utilisation, and an avalanche-versus-snowball payoff simulator that detects when a budget cannot cover accruing interest.
Insurance	Policies with premium tracking and coverage-gap analysis — life at roughly ten times annual income, health at the greater of ₹5 lakh or half of income.
Safety Net	A single 0–100 readiness score over four weighted pillars: emergency fund 35, life cover 25, health cover 25, retirement assets 15.

Overview — the answers

FEATURE	WHAT IT DOES
Dashboard	Net worth, key metrics, safe-to-spend, a generated narrative and a net-worth sparkline drawn from real daily snapshots.
Financial health	A 0–100 score across four weighted categories — Wealth 30, Protection 25, Efficiency 25, Future 20 — with the methodology shown to the user rather than hidden.
Reports	Income and expense columns, category donut, net-worth trend, over selectable windows.
Forecast PRO	Cash-flow projection from the median of completed months, net-worth projection, and what-if debt scenarios.
Insights	Spending anomalies against a trailing three-month per-category average, required SIP for a goal, and safe-to-spend as the minimum of two honest ceilings. Pure rules — no LLM, no network.

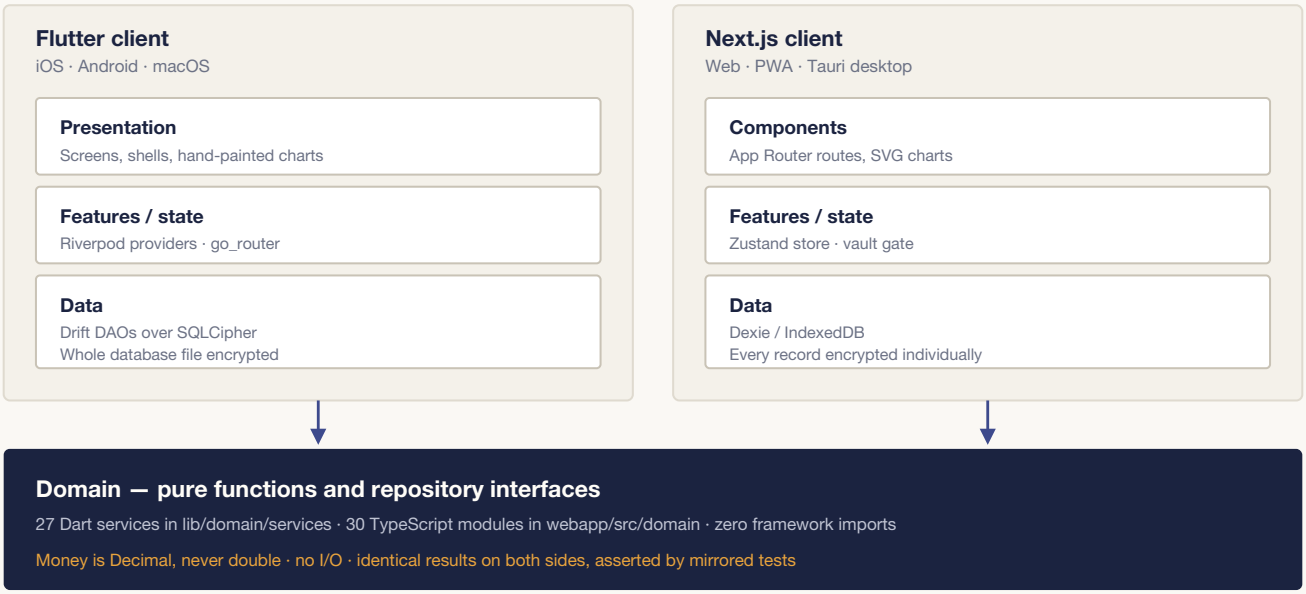
Tools and data

FEATURE	WHAT IT DOES
Bank import	One auto-detecting CSV and XLSX engine rather than a parser per bank — column aliasing plus header scanning covers HDFC, SBI, ICICI, IDFC, Kotak and Qatari banks, handling both separate debit/credit columns and single signed amounts.
Broker import	Trade and lot import with charges broken out into brokerage, STT, stamp duty, GST and other, flagging rows for review when a trade date is missing — because that date decides short versus long term.
Deduplication	Fingerprinting on date, amount, direction and normalised narration, deduping within the file as well as against the existing vault. Every import is one undoable batch.
Auto-capture PRO	Android notification-listener capture of bank alerts. The parser extracts amount, merchant and timestamp, discards the raw message immediately, and queues a draft — nothing auto-commits to the ledger.
Reconcile PRO	Statement-balance reconciliation with an explicit uncleared list as the explanation for any difference.
Diagnostics	Self-checks for undecryptable records, journal entries whose postings do not sum to zero, postings against deleted accounts, unclean money strings, lot-to-holding mismatches and stale FX rates.
Reminders	Local notifications for bills, budgets, renewals, goals and a digest — with quiet hours, cooldowns, and amounts hidden by default.
Backup and restore	An encrypted, authenticated, versioned backup file, plus a plain CSV dump and an erase-everything command. All free forever.
Vaults and profiles	Multiple vaults, each with its own PIN and encryption key, and profiles inside a vault for self, spouse or business.
Ghost mode	One toggle masks every monetary value on screen for use in public.

Section 04

System architecture

One product, two independently implemented clients, four rings each, and a domain core that imports no framework at all.



Dependencies point inward. Nothing in the domain layer knows that Flutter, React or SQLite exist.

The layered model, held identically on both clients. The value of the inward-pointing rule is concrete rather than aesthetic: because the domain layer has no I/O and no framework imports, it was portable from Dart to TypeScript by hand, and it is testable without a database, a widget tree or a browser.

Why the domain layer is framework-free

The rule sounds academic until you try to ship the same product twice. A health score that reaches into a Riverpod provider cannot be ported; a net-worth calculation that queries a DAO cannot be tested without a database. By keeping the domain as plain functions over plain entities, three things became possible at once: the logic ports across languages, it tests in microseconds without fixtures or mocks, and — the point that matters commercially — both clients can be proven to compute the same number for the same input.

The stack, concretely

CONCERN	FLUTTER CLIENT	NEXT.JS CLIENT
Language	Dart 3.10	TypeScript 5
UI	Flutter 3.38, Material with a custom design system	React 19, Tailwind v4 with CSS-native theming
State	Riverpod 2, manual provider wiring for DI	Zustand 5, one store as the whole session
Routing	go_router 17 with a vault-state redirect guard	Next.js App Router, static export
Persistence	Drift 2.2x over SQLCipher, generated by build_runner	Dexie 4 over IndexedDB
Crypto	pointycastle for PBKDF2, package:cryptography for AES-GCM, SHA-256 and Ed25519	Web Crypto for PBKDF2 and AES-GCM, @noble/ed25519 for signatures

CONCERN	FLUTTER CLIENT	NEXT.JS CLIENT
Money	package:decimal	decimal.js at 30-digit precision
Charts	Six hand-written CustomPainters	Nine hand-written SVG components
Packaging	App Store, Play Store, Mac App Store, signed DMG	Static export, PWA, Tauri 2 for Windows, macOS and Linux

DELIBERATE NON-DEPENDENCIES

There is no charting library on either side, because every chart needed a pixel-comparable twin in the other language and a library would have made that impossible. There is no web font fetched at runtime — Inter is bundled, so the app never touches the network to render. And there is no analytics, crash-reporting or telemetry SDK anywhere in either dependency tree; that absence is verified by inspection, not asserted in a privacy policy.

Section 05

Data and persistence

23	v6	FTS5	TEXT
DRIFT TABLES	SCHEMA VERSION	SEARCH INDEX	MONEY STORAGE TYPE

Money is never a floating-point number

This is the single non-negotiable rule of the codebase, and it propagates further than people expect. Money is an arbitrary-precision decimal end to end — `package:decimal` in Dart, `decimal.js` in TypeScript — and it is stored in SQLite as `TEXT` through a custom type converter, never as `REAL`. The consequence that has to be handled deliberately: because the column is text, numeric aggregation and sorting happen in Dart rather than lexicographically in SQL.

Parsing is deliberately tolerant rather than strict. Real inputs arrive with non-breaking spaces, zero-width characters, currency symbols, thousands separators and accounting parentheses for negatives. The parser recovers from all of them, warns, and returns zero rather than throwing — because a single malformed row used to take down an entire screen. Rounding rules are explicit and local: tax stays in decimal until the final step, EMI is admitted to be irrational and is computed in floating point then rounded once, and decimal division uses a fixed scale rather than an implicit one.

Two storage models, one guarantee

	FLUTTER — DRIFT OVER SQLCIPHER	WEB — DEXIE OVER INDEXEDDB
Unit of encryption	The entire SQLite database file	Each record individually
Mechanism	`PRAGMA key` issued as the first statement, using the bundled SQLCipher library	AES-256-GCM with a fresh 12-byte IV per record
Plaintext on disk	None	Only `id`, `type` and `vaultId` — the index keys
Wrong key behaviour	A probe query fails fast and raises a typed "wrong credentials, file untouched" error rather than "your file is corrupt"	The GCM authentication tag fails and the record is skipped

A detail worth naming in an interview: the SQLCipher library override is pinned to the bundled binary on every platform specifically because iOS and macOS ship their own SQLite, and letting the system library shadow the bundled one produces a database that silently writes plaintext. That failure mode is invisible — everything works, and nothing is encrypted.

The schema, and how it got to version six

Twenty-three tables plus an FTS5 virtual table with a porter tokenizer and three synchronisation triggers, in a single encrypted database; error logs live in a second, separately encrypted database so that a crash-logging bug can never corrupt the ledger. Every table carries a `vaultId` for multi-vault isolation and a `profileId` where the entity is profile-scoped.

VERSION	MIGRATION
v2	Insurance policies and daily net-worth snapshots.

VERSION	MIGRATION
v3	**Double-entry accounting.** Adds accounts, postings and pending captures, then converts every existing single-entry transaction into a balanced two-leg journal entry using deterministic account identifiers. The migration is idempotent, because a half-applied accounting migration is worse than none.
v4	**Lot-level portfolio.** Adds instruments, trades, prices, dividends, fund holdings and benchmark series. Before this, holdings were one aggregated row per symbol with a single average cost and no sector — profit and loss was not computable at all.
v5	Health score and tracked weight recorded on each snapshot — nullable, with no back-dating of scores that were never actually computed.
v6	A notification delivery log, so reminder cooldowns survive a restart.

Search

```
CREATE VIRTUAL TABLE transactions_fts USING fts5(
  txn_id UNINDEXED, content, tokenize='porter unicode61');
```

Triggers keep the index in step with inserts, updates and deletes, so search can never go stale. Query strings are escaped before reaching SQLite — that escaping is named in the threat model as the injection control, since it is the one place user text becomes query syntax.

Multi-device reconciliation without a coordinator

Two devices reconcile their encrypted backups with no server involved. The merge is last-write-wins keyed on each record's `updatedAt`, with tombstones retained for ninety days so a deletion propagates instead of being resurrected by the other device's stale copy. The function is symmetric — it runs identically on both peers and produces the same result regardless of which side runs it — and on an exact timestamp tie, delete wins.

Section 06

Security and privacy engineering

The security story here is applied client-side cryptography, not infrastructure hardening. There is no server, no account and no network trust to reason about. The whole model reduces to one sentence: on-device data must be unreadable without the user's PIN.

The key lifecycle

```
PIN + 32-byte salt  ──PBKDF2-HMAC-SHA256, 600,000 iterations──▶  256-bit AES key
```

1. **Derive.** On native the derivation runs off the UI isolate so unlock does not jank; on web it uses Web Crypto. Both are standard PBKDF2 and produce the same key from the same inputs.
2. **Use.** Mobile opens SQLCipher with the key as the very first statement, followed by a probe query that fast-fails on a wrong key. Web derives a *non-extractable* AES-GCM key object and encrypts each record with a fresh IV.
3. **Store.** Only the salt is persisted. The key itself is never written anywhere in the default flow.
4. **Purge.** On lock — five minutes idle, tab hidden, or app backgrounded — the in-memory key and all decrypted data are cleared, and routing falls back to the gate.

Verifying a PIN that was never stored

At setup the app encrypts a known token with the derived key and keeps the ciphertext. On unlock it derives a key from the entered PIN and attempts to decrypt that verifier. Because AES-GCM is authenticated, a wrong key fails the authentication tag and throws — so a wrong PIN simply cannot produce a plausible decryption, and no data is exposed in the attempt. On mobile the equivalent is the SQLCipher probe query.

What the crypto defends against

THREAT	DEFENCE
Attacker copies the database at rest	AES-256 at rest on both platforms; without the key the file is ciphertext.
Brute-forcing the PIN into the key	PBKDF2-HMAC-SHA256 at 600,000 iterations with a 32-byte salt, plus an exponential unlock lockout checked *before* the derivation runs, so an attacker cannot use the rate limiter to consume our CPU.
Tampering with ciphertext or a backup file	AES-GCM authenticated encryption; the tag is verified on every read.
Forged or corrupt backup restore	A six-gate validation: magic header, envelope version, structure, decryption, SHA-256 digest compared in constant time, then schema version.
Turning the restore screen into an oracle	A wrong PIN and a tampered file report the same error deliberately, so an attacker cannot learn which of the two they hit.
Key lingering in memory	Purged on inactivity, tab-hide and app-background.
Sensitive values leaking into logs	A sanitiser redacts any run of four or more digits before a log line is written, leaving short values like status codes intact.
Shoulder-surfing	Ghost mode masks every monetary value with one toggle.

THREAT	DEFENCE
Exfiltration from the web client	A content-security policy whose load-bearing line is `connect-src 'self'` — this client fetches nothing, so anything attempting to send a decrypted vault to another origin is blocked outright.

The decision I am most proud of: removing a key from the keychain

The original design stored the derived key in the OS keychain so biometrics could unlock without a PIN. That broke on macOS in a way that took real debugging: keychain items are bound to the code signature that created them, so every rebuild turned the app into a stranger asking for another application's secret, surfacing as `errSecMissingEntitlement (-34018)`.

The fix could have been a keychain configuration tweak. Instead I changed the model: the default path now stores only the salt and a domain-separated SHA-256 verifier of the derived key, and recomputes the key from the PIN at every unlock. Biometric unlock became opt-in, storing the key only when the user explicitly asks for it. **The result is that in the default flow there is no key at rest at all** — a security improvement that arrived through a usability failure, which is the honest way most of them arrive.

Platform hardening

- **Android** — `allowBackup="false"` and explicit data-extraction rules, because the platform default silently uploads app data. `READ_SMS` and `RECEIVE_SMS` are absent by choice; the notification listener covers the same alerts without a restricted permission.
- **macOS** — sandboxed, with only outbound network client access and user-selected file access. The network *server* entitlement is deliberately absent: the app never listens.
- **iOS and macOS** — Face ID usage strings and a declared non-exempt-encryption flag, both verified in CI so a store rejection cannot come as a surprise.
- **Keychain** — one pinned configuration used everywhere, set to first-unlock-this-device and explicitly non-synchronisable, so secrets survive a reboot but never reach iCloud or another device. Deletion overwrites before removing.

What it deliberately does not defend against

Stating this clearly is part of the security posture, not a caveat to it. A compromised device — jailbreak, keylogger, or malware running as the user — can observe the PIN or decrypted memory while the vault is open. A forgotten PIN means the data is gone; there is no recovery, no escrow and no reset, which is the price of the guarantee. Coercion is out of scope. Server-side concerns are not applicable because there is no server. And the side-channel resistance of SQLCipher, Web Crypto and pointycastle is relied upon as provided rather than independently verified.

Zero telemetry, and what replaces it

There is no analytics, crash-reporting or product-metrics SDK in either client. The reasoning is specific: a stack trace from a finance app is a description of what the user was doing when it broke, and I have no business holding that. The consequence is that the only crash report I will ever see is one a user chooses to send from Settings.

What stands in its place is a local crash guard that installs handlers for framework errors, uncaught platform errors and async gaps, and buffers up to fifty scrubbed records in memory — because a crash loop can throw thousands of times a second, and the first few explain the last few. Once the vault opens, the buffer drains into an encrypted log database with a thirty-day retention. Records are scrubbed *before* entering the buffer, so nothing sensitive sits in memory either. The replacement error screen tells the user the one thing they actually need to know: their data is safe and unchanged.

Section 07

The financial engine

Twenty-seven pure Dart services and thirty TypeScript modules, none of which perform I/O. This is the part of the project I would most want to be asked about.

Lot-level portfolio and FIFO matching

Sells consume the oldest open lots first, ordered by trade date with the identifier as a deterministic tie-break so the same input always produces the same match. Proceeds are computed net of the sale's charges, pro-rated so a partial match carries its share. Two sign conventions matter and are easy to get backwards: **charges add on a buy and subtract on a sell**. Reversing that is the standard way profit and loss ends up reading better than reality.

Selling more than was ever bought is skipped rather than turned into a negative or short lot. FIFO is deliberately not configurable, on the grounds that choosing a method you are not entitled to produces a return you cannot file.

> **THE INVARIANT THAT ANCHORS THE WHOLE MODULE**
>
> A roll-up along *any* dimension — sector, industry, market cap, asset group, asset type, instrument, currency — must sum exactly to the portfolio total. Unclassified values become an explicit bucket and are never dropped. This is asserted in both the Dart and the TypeScript test suites, and the rule when it fails is stated in the source: if the reconciliation test breaks, the roll-up is wrong, not the test.

Honest nulls — a theme rather than a detail

A recurring design decision across the engine is to return *nothing* rather than a plausible-looking zero:

- Profit-and-loss percentage is `null`, rendering as an em dash, when cost is zero — never 0%.
- XIRR returns `null` when there is no sign change or fewer than two cash flows, rather than a fabricated rate.
- Concentration is `null` on an empty book, not 0.
- Safe-to-spend returns a typed gap — "no income recorded", "no history" — rather than inventing an allowance.
- The financial-health grade is withheld entirely below fifty points of tracked weight, because a letter grade computed from a quarter of the picture is a lie with a serif on it.
- Unpriced holdings are counted at cost, contribute zero profit and loss, and every roll-up row carries an `unpricedCount` so the interface can disclose that its own number is understated.

XIRR by bisection

Money-weighted return is found by bisecting the net-present-value function over a rate range of −99.99% to 1000%, with 200 iterations and a tolerance of 1e-7. Portfolio flows are buys negative, sells and dividends positive, and current market value as a terminal flow. Bisection rather than Newton–Raphson is a deliberate robustness choice: Newton converges faster but can diverge on the irregular, sign-changing cash-flow series a real portfolio produces, and a wrong return is worse than a slow one.

Double-entry accounting

Postings are debit-signed and every journal entry sums to zero. Net worth is computed from asset and liability balances plus the market value of holdings, structured so nothing is counted twice — transfers debit the destination and credit the source, so they cancel. The balance sheet derives two lines the ledger does not store: retained earnings, as cumulative income minus expense, since income and expense accounts are never closed; and opening capital, as the contra for

account opening balances. With both present, assets plus liabilities plus equity is identically zero — which means any non-zero result is real damage, and that is exactly what the diagnostics screen checks.

Capital-gains tax

Rules are a versioned data set selected *by disposal date*, so a sale made before July 2024 is not taxed under rules that did not exist when it happened. Where the short-term rate is slab-based the engine reports it as slab rather than guessing a number, and surfaces slab-rate gains separately so the headline figure cannot understate the bill. The most interesting piece of modelling is the annual exemption:

```
// The ₹1.25 lakh long-term exemption is ONE allowance per financial year,
// not per holding. It is pooled and spent largest-gain-first, which is both
// optimal for the user and deterministic — the same book always produces
// the same allocation.
```

Per-asset treatment is modelled rather than approximated: foreign equity is not eligible for the concessional long-term regime because no securities transaction tax was paid; a sovereign gold bond redemption is a maturity event and not a disposal; fixed-income instruments with no quoted price are valued as principal plus accrued interest, compounding quarterly for cumulative deposits per Indian convention, with accrual stopping at maturity.

Weighted scoring, and the choice not to punish absence

The financial-health score runs over four categories — Wealth 30, Protection 25, Efficiency 25, Future 20 — each composed of weighted metrics. The design decision that makes it usable:

```
// A metric's value is null — never 0 — when the user has not tracked it.
// Untracked metrics leave the DENOMINATOR rather than scoring zero.
const score = Math.round((tracked.reduce((s, c) => s + c.score, 0) / trackedWeight) * 100);
const grade = trackedWeight >= MIN_GRADABLE_WEIGHT ? gradeOf(score) : null;
```

A user who has not entered insurance is not failing at insurance — they are not tracking it, and a score that conflates the two teaches people to distrust the number and stop looking. Related touches: a flat net worth scores 0.33 on trajectory rather than zero, because holding steady is not failure; the debt component is multiplied by 0.85 when any interest rate exceeds 24%; goal pace excludes emergency-fund goals because those are Protection's business; and Protection reuses the Safety Net components directly rather than recomputing them, so the two scores cannot disagree.

Colour as a computed property

The allocation charts have to separate fourteen asset types, which is far more categories than can be reliably distinguished by hue. Rather than assert that a palette is colour-blind safe, the repository computes it: Machado (2009) colour-vision-deficiency matrices applied to *linear* RGB — applying them to gamma-encoded values inflates the apparent separation and would let a failing palette pass — with CIE76 colour difference in Lab space and WCAG relative luminance checked at 3:1 for graphic objects. The palette is authored in OKLCH per theme, and a test asserts the resulting bounds.

That analysis produced a counter-intuitive rule now enforced in the code: **the allocation chart must never sort its slices by value.** Sorting makes colour adjacency depend on the data, and re-sorting drops the worst adjacent-protanopia colour difference from 9.1 to 3.2 — below the threshold at which two neighbouring slices remain distinguishable. Fixed group order plus a legend is not decoration; it is load-bearing.

Import parsing

Bank statement import is one auto-detecting engine rather than a parser per bank, because a parser per bank is a maintenance obligation that grows without bound. Column aliases plus a header-row scan cover the major Indian banks; the delimiter is sniffed by *column-count consistency* rather than raw character frequency, because a narration full of commas will otherwise beat a real tab. Both separate debit/credit columns and single signed amounts with a direction indicator are handled, and a default direction is applied only when the file states none at all. XLSX support loads its parser lazily, since it is a 400 KB dependency most users never trigger.

Section 08

Cross-platform parity

The hardest problem in this project was not any single algorithm. It was shipping the same product twice, in two languages, without the two copies quietly drifting apart.

The naive answers both fail. A shared package means one language, which means Flutter-on-web and the compromises that come with it. A code generator means maintaining a code generator. What worked instead was to make disagreement *fail a test*, using three mechanisms.

1 · Byte-identical fixtures

The financial-health test cases exist as a single JSON file duplicated byte for byte on both sides and loaded by both suites. Neither implementation can be adjusted to make its own tests pass without the other's tests failing on the same data. The same pattern covers the notification scheduler and the licence-key format.

2 · Cross-language assertions on shared inputs

The Safety Net score is asserted at exactly 100, 25, 50 and 85 for four shared fixtures — in `test/unit/safety_net_test.dart` and in `webapp/src/domain/safetyNet.test.ts`. The same is true of the portfolio roll-up reconciliation invariant and the narrative generator's shared banned-phrase list, which enforces "describe, never advise" identically in both languages.

3 · A specification file that drives both clients

The free-versus-Pro feature map lives in `docs/pro-gates.json` and is loaded by the Dart gate tests, the TypeScript gate tests, and a parity test that checks the web navigation's Pro flags against it. The two clients cannot disagree about what money buys, because neither of them is the source of truth — the specification is.

A five-way version-consistency check runs first in CI, because the version number is a compatibility gate in the backup header — and it had already drifted once, with the app at 1.0.0 while the web and desktop builds said 0.1.0.

Where parity is deliberately not attempted

The Flutter client is the lot-level ledger; the web client still stores aggregated positions and therefore shows unrealised profit and loss only. That is a known and stated gap rather than a hidden one. Deciding *where* parity stops is as much a part of the strategy as enforcing it where it holds — chasing full parity on a feature only one platform's users need would have cost the release.

Section 09

Decisions and trade-offs

The decisions worth defending, each with the alternative that was rejected. These are the questions an interviewer should be asking.

DECISION	REJECTED ALTERNATIVE	REASONING
No server, at all	A thin sync or licence-validation backend	A server implies accounts, a database of other people's finances, a breach to disclose and a bill that outlives interest in the project. Removing it shrank the threat model to one sentence.
Money as arbitrary-precision decimal in TEXT columns	Integer minor units, or floating point	Floating point is disqualified outright. Minor units work until you divide — for interest accrual, allocation and tax, decimal keeps precision where integers force premature rounding. The cost, accepted knowingly, is that aggregation happens in application code.
No key at rest by default	Keeping the derived key in the OS keychain for convenience	Re-deriving from the PIN on every unlock costs about 45 ms and removes an entire class of at-rest compromise. Biometric convenience became opt-in rather than default.
600,000 PBKDF2 iterations	A cheaper KDF for a snappier unlock	The cost is the point — it is the per-guess price an offline attacker pays against an exported backup. 45 ms on web is the unlock latency budget, spent deliberately.
Three storage contracts frozen forever	Renaming them during the rebrand for tidiness	The verifier token, the IndexedDB database name and the backup magic bytes are compared against existing user data. Renaming any of them locks every existing vault out permanently with no recovery. They keep their old names, and the source says why.
Bisection for XIRR	Newton–Raphson	Newton converges faster but can diverge on the irregular sign-changing series real portfolios produce. A slow correct answer beats a fast wrong one, and returning null beats both when the series does not bracket a root.
Untracked metrics leave the denominator	Scoring untracked areas as zero	Scoring absence as failure teaches users to distrust the score and stop looking at it. Withholding the grade below fifty points of tracked weight is more honest than grading a quarter of the picture.
Fixed slice order in allocation charts	Sorting slices by value, as is conventional	Sorting makes colour adjacency data-dependent; the measured worst-case protanopia separation drops from 9.1 to 3.2 and neighbouring slices stop being distinguishable. Measured, not assumed.

DECISION	REJECTED ALTERNATIVE	REASONING
Unknown asset type throws	Falling back to a default type	The old fallback silently mis-taxed anything unrecognised. A tolerant variant exists for the places that genuinely need one; the strict version is the default because wrong tax is worse than an error.
Three data files left empty on purpose	Filling them with plausible values so features light up	Mutual-fund overlap needs real asset-management-company disclosures and index comparison needs real historical closes. Inventing either produces confident, wrong numbers — the worst possible output for a finance tool.
Captured bank alerts never auto-commit	Writing parsed transactions straight to the ledger	A parser that is 95% right silently corrupts a ledger. Captures queue in a review inbox, and the raw message is discarded the instant it is parsed.
No charting library	fl_chart plus Recharts	Two libraries with different rendering models cannot be made to agree pixel for pixel, and parity was the whole strategy. Six painters and nine SVG components is more code and less risk.

Comments as an incident log

A pattern I adopted deliberately and would carry to any team: where a piece of code exists to prevent a specific defect, the comment names that defect, including its error code. The keychain configuration names `errSecMissingEntitlement (-34018)`. The SQLCipher executor names the system-SQLite shadowing pitfall that produces a silently plaintext database. The service worker names the bug where returning `undefined` from a fetch handler rendered the browser's error page instead of the app. The icon check in the release pipeline names App Store rejection `ITMS-90717`.

The value is that these comments survive me. Six months later the reason a line looks strange is written next to it, and nobody "cleans it up".

Section 10

Testing, CI and measured performance

1,392	128	3	2
TEST CASES	TEST FILES	CI PIPELINES	COMMITTED BENCHMARKS

SUITE	FILES	CASES	NOTES
Dart — unit	46		Domain services, pure and fast
Dart — integration	9		Repository and database paths
Dart — widget	6		Screen behaviour
Dart — golden	3	561	Pixel baselines; excluded from CI as host-font dependent
Dart — on-device	3		Encryption round-trip, macOS keychain, notification delivery
Dart — benchmark	1		Prints real numbers, asserts only loose bounds
TypeScript — Vitest	60	831	Domain, library, component and route-level tests

The three test types worth talking about

Invariant tests check that the functions agree with *each other*, not that each one is individually correct. The integrity suite states the problem it exists for: every screen can be individually right and still disagree with the one next to it. Its fixtures are built by the application's own posting functions rather than written by hand — a fixture with hand-written postings can be made to balance by construction and would prove nothing.

Write-path tests cover the mutations rather than the reads. One of them exists because a transfer written over an existing transaction had to retire that transaction, or a single identifier would end up owning two records sharing one pair of ledger legs.

A critical-path test walks the whole seam: CSV in, parse, valuation, reconciliation, backup, restore into a *second* vault. It was added after a day of real use produced a run of integration bugs while every unit test stayed green — the units were fine, the seams between them were never exercised.

Continuous integration

PIPELINE	WHAT IT DOES
ci.yml	On every push and pull request. A cheap version-consistency job runs first across five declaration sites; then the web job runs Vitest and a static build as hard gates with lint advisory; then the Flutter job runs codegen, analyze and the test suite.
release.yml	On a version tag. A four-way desktop matrix — Windows installer and MSI, macOS Apple Silicon and Intel built separately rather than as a universal binary, Linux AppImage and deb pinned to an older Ubuntu so the result runs on current distributions. Opens a draft release.
release-mobile.yml	On a version tag. Asserts the tag matches the manifest *before* a twenty-minute build. Restores the signing keystore from secrets, writes it to disk only for the job's life and removes it unconditionally. Then unzips the built bundle and asserts against the compiled manifest that internet permission is present, backup is disabled, and the SMS permissions are absent.

That last check is a regression test for a bug that shipped once: minification and manifest merging are exactly where a permission or a missing capability quietly reappears, and neither of them fails the build on its own.

Measured, not claimed

The performance numbers below come from committed scripts anyone can re-run, and they are recorded as measurements on a development host rather than as guarantees.

PRIMITIVE	PATH	MEASURED
PBKDF2-HMAC-SHA256, 600k → 256-bit key	Web Crypto	≈ 45 ms
PBKDF2-HMAC-SHA256, 600k → 256-bit key	Pure Dart, JIT	≈ 1.2 s
AES-256-GCM encrypt, transaction-sized record	Web Crypto	≈ 0.018 ms
AES-256-GCM decrypt	Web Crypto	≈ 0.022 ms
FTS5 search across 10,000 transactions	SQLite via Drift	≈ 0.13 ms

The 600,000-iteration cost is intentional; ~45 ms is the unlock latency budget, spent for brute-force resistance. The pure-Dart figure is the just-in-time test-VM path — release builds are ahead-of-time compiled and faster, and Flutter on web uses Web Crypto directly.

Section 11

From project to product

In August 2026 Khazana stopped being a portfolio project and became something I intend to sell. The interesting constraint was monetising it without breaking the one rule the whole architecture rests on.

The commercial shape

- **One brand, both clients.** The Flutter app goes to the App Store, Play Store and Mac App Store. The web and Tauri desktop builds sell directly through a merchant of record, so they own Indian GST and EU VAT obligations rather than me.
- **A one-time unlock, not a subscription.** Khazana Pro at roughly ₹999 or $19.99, as a non-consumable with Family Sharing. A subscription would require a server to check, which is exactly the thing that must not exist.
- **Published as an individual.** The consequence is concrete: Google Play requires a twelve-tester, fourteen-day continuous closed test before production access. That is the critical path to launch, and it is not a code problem.

Licensing with no licence server

Direct-sale licence keys are Ed25519-signed and verified entirely offline. The key is a 76-byte payload encoded to about a hundred characters: a format version, a product byte, an issue date, eight bytes of order-reference hash, and a 64-byte signature over the rest. Keys are minted in batches by a local command-line tool and uploaded as a pre-signed pool to the merchant — so a purchase hands out a key from the pool and no validation endpoint ever exists.

Two details in that format are deliberate and worth defending. There is **no expiry field**, because offline verification would have to trust the device clock, and a user who changes their date should not lose the software they bought — this is the strongest structural reason Pro can never become a subscription without a server. And there is **no name, email or device identifier**, only a hash of the order reference, so the paywall can display "Licence …4F2A" for support purposes while holding no personal data at all.

Two rules that constrain every gate

> NON-NEGOTIABLE
>
> **1. Backup export, restore, plain CSV dump and erase-everything are free forever.** Never hold a user's own data hostage to a purchase.
> **2. The ledger is never capped.** No transaction, account, budget or holding limits in any tier — because a capped ledger reports wrong totals, and every derived figure inherits the lie.

These are enforced structurally rather than by discipline. The always-free set is checked *before* the entitlement is even read: the ordering is the enforcement. Only two counted allowances exist in the free tier — one profile and one committed import, and that import is undoable.

Entitlement engineering

- One interface with three implementations: StoreKit non-consumable, Play Billing one-time purchase, and offline signed keys for web and desktop.
- Entitlement persists outside the vault, so it survives an erase-all-data and can be read before unlock — a paywall that needs the PIN to know whether to show itself is a paywall that shows itself at the wrong time.
- Revocation requires two absent store queries at least seven days apart. An interactive "restore purchases" can never revoke. Someone on a plane does not lose the software they paid for.

- The Pro gate renders the real content blurred and inert behind the upgrade card rather than redirecting, so the user can see what they would get.
- The licence-key entry UI is removed at *compile time* from App Store builds rather than hidden behind a runtime condition, because App Store guideline 3.1.1 prohibits pointing at outside purchase mechanisms — and a hidden branch is still shipped code.

One limitation is stated on the paywall before purchase rather than discovered afterwards: an app-store purchase and a direct licence key are separate entitlements. Linking them across ecosystems needs an account server, and there is not going to be one.

Section 12

Metrics, limitations and roadmap

By the numbers

MEASURE	VALUE
Hand-written source and tests	≈ 85,000 lines
Dart — application code (excluding generated)	35,703 lines across 224 files
TypeScript and TSX — web client	39,109 lines
Dart tests	9,632 lines
Generated Drift code (not counted above)	≈ 18,700 lines
Tracked files in the repository	761
Domain services / modules	27 Dart · 30 TypeScript
Encrypted database tables	23 + FTS5 index
Screens and routes	27 Flutter · 31 web
Automated tests	1,392 across 128 files
Platforms targeted	iOS, Android, macOS, Web, Windows/Linux desktop
Commits · active days · elapsed	83 · 17 · 9 weeks
Servers, accounts, telemetry endpoints	0

Built between 22 June and 25 August 2026, single-handed, alongside full-time work — which is why the commit history is seventeen dense days rather than a steady drip. The largest single stretch was fifteen commits on 1 August.

What does not work yet

The repository maintains a numbered defect log with status markers, and several entries have been fixed on the current hardening branch. These are the open ones, reproduced rather than laundered:

ITEM	STATUS
Transaction editing OPEN	Editing exists on the web client but not yet as a dedicated Flutter screen.
Desktop content policy OPEN	The Tauri wrapper does not carry the content-security policy the hosted web build enforces.
Web lot-level parity OPEN	The web client stores aggregated positions and therefore shows unrealised profit and loss only.
Android capture path OPEN	The Kotlin notification-listener bridge is unverified on a physical device.
Encryption tests not in CI OPEN	The on-device tests that guard encryption at rest need a real device, so they are in the manual release checklist rather than the pipeline — which means the tests guarding the product's core promise do not run automatically.
Unsigned desktop installers OPEN	Windows SmartScreen and macOS Gatekeeper will warn. Stated in the release notes rather than left for the user to discover.

ITEM	STATUS
A beta dependency OPEN	Secure storage is pinned to a beta release. A beta is not a shipping dependency for a paid security product; it is a launch blocker on the checklist.
Web lint debt OPEN	Pre-existing rule violations under the newest ruleset. Lint is advisory in CI with the reason recorded; tests and the build are hard gates.
No pagination anywhere OPEN	Defensible at local-first data volumes, but it is the clearest remaining performance gap.
One golden test OPEN	Font rendering differs between hosts; goldens pass only on the machine that generated them, so they are excluded from CI.

Closed since the previous revision of this document, each with a test that would catch its return: receipt attachments are now AES-256-GCM sealed under the vault key and purged with it, and they round-trip through the repository; recurring and imported transactions now write through the ledger writer, pinned by a test asserting that nothing outside the dependency wiring reaches the transaction DAO directly; and the backup restore path is implemented, reachable and round-tripped, including refusing a wrong key, a corrupted file and a backup from a newer build. The first of those was the item this section previously called the clearest violation of the project's own encryption contract.

What comes next, in order

1. Upgrade off the beta secure-storage dependency and verify the Android capture path on hardware.
2. Start the twelve-tester closed track, since fourteen continuous days is the longest lead time in the plan.
3. Code signing for desktop; content-security policy for the Tauri wrapper.
4. Optional peer-to-peer sync between a user's own devices, using the merge function that already exists and is already tested.

Appendix A

LinkedIn copy

Written to be posted as-is. Adjust the closing line to match whether you are hiring, looking, or simply shipping.

SHORT POST — ANNOUNCEMENT

I spent two months building the personal finance app I actually wanted to use.

Khazana keeps every rupee of your financial data encrypted on your own device. No backend. No account. No telemetry. Your PIN derives the encryption key, the key is never stored, and there is no recovery — because there is nobody who could recover it.

Under that vault: double-entry accounting, lot-level portfolio tracking with FIFO matching, XIRR, an Indian capital-gains engine versioned by disposal date, EMI payoff simulation, and a financial-health score.

It ships as two independent clients — Flutter for iOS, Android and macOS, and Next.js for web and desktop — sharing one framework-free financial engine written twice and kept honest by mirrored tests. ~85,000 lines, 1,392 tests, 23 encrypted tables, zero servers.

Writing up the architecture in detail. Happy to talk about any of it.

#Flutter #NextJS #Cryptography #FinTech #SoftwareArchitecture

LONG POST — THE BUILD STORY

Every personal finance app I tried wanted one of two things: my banking credentials, or a copy of my complete financial life on their servers.

So I built the one that asks for neither.

Khazana is offline-first and serverless by architecture, not by configuration. I held one rule for the entire build: no server, ever. Not a light one, not "just for sync", not "just to validate licences". That single refusal forced better engineering everywhere else.

Three things it forced:

→ **Sync without a coordinator.** Two devices reconcile encrypted backups through a deterministic last-write-wins merge with 90-day tombstones. The function is symmetric — same result whichever side runs it.

→ **Licensing without a licence server.** Ed25519-signed keys, minted offline in batches, verified locally with no network call. No expiry field, because offline verification would have to trust the device clock. No email or device ID, only a hash — so the paywall shows "Licence …4F2A" and holds zero personal data.

→ **A threat model I can state in one sentence:** on-device data must be unreadable without the user's PIN. No JWT, OAuth, RBAC, CORS or CSRF surface exists to get wrong, because no endpoint exists.

The hardest problem wasn't cryptography. It was shipping the same product twice — Flutter and Next.js — without the two copies drifting apart on what a number means. The answer was to make disagreement fail a test: byte-identical JSON fixtures loaded by both the Dart and the TypeScript suites, and cross-language assertions pinning the same inputs to the same outputs on both sides.

My favourite decision came out of a bug. I had been storing the derived key in the OS keychain for biometric unlock. On macOS that broke on every rebuild, because keychain items are bound to the code signature that created them. I could have tweaked the configuration. Instead I changed the model: store only the salt and a verifier, re-derive from the PIN each time, make biometrics opt-in. The result is that in the default flow there is now no key at rest at all.

A security improvement that arrived through a usability failure. Most of them do.

~85,000 lines, 1,392 tests, 23 encrypted tables, 5 platforms, 0 servers. Full technical write-up in the comments.

HEADLINE

Software Engineer · Building Khazana, an offline-first encrypted finance app | Flutter · Next.js · Applied Cryptography

ALTERNATIVE HEADLINE

I build software that doesn't need your data on my servers · Flutter · TypeScript · Local-first architecture

ABOUT SECTION

I build products where the architecture reflects a position, not just a requirement. Most recently that is Khazana — an offline-first personal finance app with no backend, no account and no telemetry, where every rupee of data is encrypted on the user's own device behind a PIN that is never stored.

It ships as two independent clients, a Flutter app and a Next.js web and desktop app, over one framework-free financial engine covering double-entry accounting, lot-level portfolio analytics with FIFO matching, XIRR, Indian capital-gains estimation and weighted financial-health scoring. About 85,000 lines and 1,392 automated tests, including cross-language tests that make the two clients provably agree.

I care most about the decisions that are hard to reverse: how money is represented, where the trust boundary sits, and what a system promises when it has no server to fall back on.

Appendix B

Website and portfolio copy

HERO

Khazana

Your money, on your device, encrypted, and nowhere else.

An offline-first personal finance and wealth app with no backend, no account and no telemetry. Double-entry accounting, lot-level portfolio analytics and a real tax engine — behind a PIN that never leaves the device.

FEATURE GRID — SIX TILES

Encrypted at rest. AES-256 on every platform, keyed by PBKDF2-HMAC-SHA256 at 600,000 iterations. The key is derived from your PIN and never stored.

No server, by design. No account to create, no data to breach, nothing that stops working when someone stops paying a hosting bill.

Real accounting. Double-entry postings where every journal entry sums to zero — so net worth is derived, not guessed.

Portfolio that computes. Lot-level FIFO matching, XIRR, sector-wise profit and loss, and capital-gains estimation under rules versioned by disposal date.

Money that is exact. Arbitrary-precision decimal end to end, stored as text. Totals never drift by a rounding cent.

Two apps, one engine. Mobile and web compute identical numbers — enforced by tests that fail if the two implementations ever disagree.

TECHNOLOGIES

Flutter · Dart · Riverpod · go_router · Drift · SQLCipher
Next.js 16 · React 19 · TypeScript · Zustand · Dexie · Tailwind
Tauri 2 · Rust shell
Web Crypto · AES-256-GCM · PBKDF2 · Ed25519 · SHA-256
SQLite FTS5 · IndexedDB · decimal arithmetic
GitHub Actions · Vitest · flutter_test · golden tests

AT A GLANCE

≈85,000 lines hand-written
1,392 automated tests
23 encrypted tables, 6 schema migrations
27 + 30 pure domain modules
5 platforms
0 servers
Solo build, 9 weeks

PROJECT BLURB — 150 WORDS

Khazana is an offline-first personal finance and wealth application built on a single constraint: no server, ever. Every rupee of data is encrypted at rest on the user's own device — SQLCipher on mobile, per-record AES-256-GCM on web — behind a key derived from a PIN that is never stored and has no recovery path.

It ships as two independently implemented clients, a Flutter app for iOS, Android and macOS and a Next.js app for web and desktop, sharing one framework-free financial engine: double-entry accounting, lot-level portfolio tracking with FIFO matching, XIRR, an Indian capital-gains engine versioned by disposal date, EMI payoff simulation and weighted financial-health scoring.

Keeping two implementations honest was the central engineering problem. The answer was to make disagreement fail a test — byte-identical fixtures and cross-language assertions across roughly 85,000 lines and 1,392 tests.

ONE-LINE VERSIONS

An encrypted, serverless personal finance app — two clients, one financial engine, zero backends.

Personal finance software that cannot leak your data, because it never has it.

Local-first finance: AES-256 at rest, double-entry accounting, lot-level portfolio analytics, no account required.

Appendix C

Interview question bank

Sixteen questions this project invites, with answers already thought through. Answer in your own words — the point is that the reasoning is ready, not the phrasing.

Architecture

Why no backend? Isn't that just avoiding the hard part?

It moves the hard part rather than avoiding it. Removing the server removed an entire threat surface — no JWT, OAuth, RBAC, CORS or CSRF to get wrong — but it made three things genuinely harder: multi-device reconciliation with no coordinator, licence verification with no validation endpoint, and migrations that run on a database I can never inspect. I solved those with a symmetric last-write-wins merge with tombstones, offline Ed25519 signatures, and idempotent migrations. If the role needs distributed systems experience I would talk about different work — but I would not describe this as the easy path.

Why two codebases instead of Flutter Web?

Flutter Web renders to canvas, which is heavy to load and awkward to make properly accessible. I wanted the web client to be real DOM, statically exported, installable as a PWA and wrappable in Tauri. The cost is duplicated UI. The benefit I did not anticipate is that having to write the financial logic twice forced it out of the UI layer completely — which turned out to be the best structural decision in the project.

Walk me through what happens when a user opens the app.

Cold start shows a lock gate, never data. The router's redirect sends every application route to the unlock screen unless the vault state machine says unlocked. The user enters a PIN; it is stretched with PBKDF2-HMAC-SHA256 at 600,000 iterations against a stored 32-byte salt — off the UI isolate on native so it does not jank. The PIN was never stored, so correctness is checked by decrypting a verifier token encrypted at setup; AES-GCM is authenticated, so a wrong key fails the tag and throws. On success the key is held only in memory, records are read and decrypted, filtered to the active profile, and handed to the store. On lock the key and all decrypted data are cleared.

Data and correctness

How do you handle money, and why not integer paise?

Arbitrary-precision decimal end to end, stored as text, never floating point. Integer minor units are a perfectly good answer and I considered them — they fail for me at division. Interest accrual, allocation percentages and tax computation all divide, and integers force you to round early and repeatedly. Decimal lets me carry precision to the last step and round once, deliberately, where the rule says to. The cost I accepted is that aggregation and sorting happen in application code rather than in SQL, since the column is text.

How do you know your numbers are right?

Three layers. Unit tests for each pure function. Invariant tests that check the functions agree with each other — a roll-up along any dimension must sum exactly to the portfolio total, and assets plus liabilities plus equity must be identically zero. And a critical-path test that walks CSV import through valuation, reconciliation, backup and restore into a second vault. I added that third layer after a day of real use produced a run of integration bugs while every unit test stayed green; the units were fine and the seams between them had never been exercised.

Explain XIRR and why you implemented it the way you did.

XIRR is the discount rate at which the net present value of an irregular series of cash flows is zero — money-weighted return, so it accounts for when you invested, not just how much. I find it by bisection over a rate range of about −99.99% to 1000%, 200 iterations, tolerance 1e-7. Newton–Raphson converges faster but can diverge on the irregular sign-changing series a real portfolio produces. And when the series has no sign change, or fewer than two flows, I return null rather than a number — a fabricated return is worse than no return.

You have two implementations of the same logic. How do they not drift?

By making drift a test failure. The health-score cases are one JSON file duplicated byte for byte and loaded by both suites, so neither side can be tuned to pass without breaking the other. The Safety Net score is asserted at the same four values in both languages on the same fixtures. And the free-versus-Pro feature map lives in a specification file that both clients' gate tests read, so neither client is the source of truth. There is also a version-consistency check that runs first in CI across five declaration sites, because the version is a compatibility gate in the backup header and it had already drifted once.

Security

How do you know it's actually secure? You're not a security engineer.

I would not claim it is audited. What I would claim is that the model is small enough to state precisely and that each control is written down with the threat it addresses. There is no server, so the whole question is whether on-device data is unreadable without the PIN. Standard primitives, used conventionally: PBKDF2 at 600,000 iterations, AES-256-GCM with a fresh IV per record, SQLCipher for whole-file encryption. The parts I would want a reviewer to look at hardest are the backup envelope validation and the lockout logic. And I document what it does not defend against — a compromised device, a forgotten PIN, coercion — because a security claim without a stated boundary is marketing.

Why 600,000 iterations? Users hate slow unlocks.

The cost is the product, not a side effect. It sets the per-guess price for an attacker running offline against an exported backup file. About 45 ms on web is a latency I am willing to spend for that, and it is measured rather than assumed — there is a committed benchmark. I also enforce a minimum PIN length at setup and an exponential lockout, and the lockout is checked *before* the derivation runs so an attacker cannot use the rate limiter to burn our CPU.

No password recovery at all? That's a support nightmare.

It is, and it is the correct trade-off for this product. Recovery requires either an escrowed key or a server that can reset something — and both re-introduce the exact trust relationship the app exists to avoid. What I owe the user instead is clarity: it is stated at setup, and backup export is free forever precisely so that the user's own encrypted copy is always their recovery path. The availability risk is real and I name it in the threat model rather than discovering it in reviews.

Judgement

What was the hardest bug?

The macOS keychain failure — see the STAR story below. The honourable mention is subtler: the web client's base64 encoder used spread-argument character conversion, which throws a range error past roughly a hundred thousand elements. It worked in every test and failed on the one thing that matters — a real, full-sized vault backup. The lesson I took is that the code path handling the largest input is often the one no test ever gives a large input to.

What would you do differently?

Three things. First, I would build the double-entry ledger from day one instead of migrating to it at schema version three — the migration that converts every existing transaction into balanced postings was avoidable work created by my own earlier shortcut. Same story for lot-level holdings at version four: the original aggregated model made profit and loss literally uncomputable. Second, I would put the cross-language fixtures in place before writing the second implementation rather than after. Third, I would set up CI before feature work, not on the same branch as it.

This is a solo project. How do I know you can work in a team?

Fair question, and I would point at the artefacts rather than argue. I wrote an architecture document, a threat model, and a screen-by-screen inventory that includes a numbered defect log with honest status markers — including entries that say the product is currently wrong. I enforce decisions in CI rather than by memory. And comments in the codebase name the specific defect they exist to prevent, with error codes, so the reason a strange line exists survives me. Those are all habits for other people's benefit; I just happened to be the only other person.

Why should I believe your test numbers?

Count them: the test files and cases are in the repository and greppable, and the performance figures come from committed benchmark scripts you can re-run rather than numbers I typed into a document. That distinction is deliberate — the README says the numbers are measured, not asserted from a specification, and records that they are host-dependent.

What's the weakest part of the codebase?

Until recently it was attachments — the one place the encryption contract was not honoured, and I treated it as the priority precisely because it contradicted the product's central claim rather than because anyone had complained. Receipts are now sealed with AES-256-GCM under the same key as the database, the plaintext never touches disk even briefly, and a file that fails authentication raises instead of quietly rendering blank, because silent failure would hide tampering. What I would point at now is that there is no pagination anywhere: defensible at local-first data volumes, but it is the clearest remaining performance gap, and it is the kind of thing that is cheap to fix early and expensive once someone has a decade of transactions.

How did you decide what not to build?

Mostly by refusing to fake data. Three data files in the repository are deliberately empty: mutual-fund overlap analysis needs real asset-management-company disclosures, and index comparison needs real historical closes. I could have populated both with plausible values and shipped features that look complete. For a finance tool, confident wrong numbers are the worst possible output — worse than a missing feature, because the user cannot tell. Same reasoning for statement parsing of password-protected consolidated account statements, and for broker API sync, which would send the user's portfolio to a third party.

Three STAR stories

1 · A security improvement that came from a usability bug

Situation. Biometric unlock stored the derived encryption key in the macOS keychain. Every rebuild during development failed vault creation with `errSecMissingEntitlement (-34018)`.

Task. Make biometric unlock work reliably across rebuilds and signing configurations.

Action. The root cause was that keychain items are bound to the code signature that created them, so each rebuild made the app a stranger asking for another application's secret. Rather than tune the keychain configuration, I questioned whether the key needed to be there at all. I changed the default path to store only the salt and a domain-separated SHA-256 verifier, re-deriving the key from the PIN on every unlock — about 45 ms — and made biometric key storage explicitly opt-in. I also consolidated five duplicated keychain configurations, all of which were wrong, into one pinned definition.

Result. The bug went away, and the default flow now has no encryption key at rest anywhere. A usability failure produced a materially better security posture, which is the honest way most improvements arrive.

2 · Discovering a feature was built on an impossible data model

Situation. Users wanted sector-wise profit and loss on the portfolio. Holdings were stored as one aggregated row per symbol with a single average cost, no sector, no price date and no charges.

Task. Ship sector P&L — and first, work out whether it was even computable.

Action. It was not. Realised profit and loss requires knowing which specific lots a sale consumed, and an average-cost row has destroyed that information irreversibly. I designed a lot-level model — instruments, trades, prices, dividends — wrote FIFO matching with deterministic tie-breaking, and migrated the schema. Crucially, I wrote the reconciliation invariant first: a roll-up along any dimension must sum exactly to the portfolio total, asserted in both languages. I also made unpriced holdings count at cost while exposing a count, so the interface can disclose that its own figure is understated rather than quietly presenting it as complete.

Result. Sector, industry, market-cap and asset-class roll-ups that reconcile exactly, plus XIRR and realised-gain tax estimation that were impossible under the old model. The lesson I carry from it: when a feature request seems disproportionately hard, check whether the data model can express the answer at all before estimating the work.

3 · A restore that wrote 367 perfectly good records where nothing could see them

Situation. Restoring a backup onto a second device appeared to succeed and produced an empty app.

Task. Find out why intact, correctly decrypted data was invisible.

Action. The restore was writing records verbatim, including their original profile identifiers. The receiving device had its own profile with a different generated identifier, so the reload filter — which scopes records to the active profile — matched nothing. The data was all there and no screen could see it. I rewrote restore to reconcile profiles by name and kind, remap incoming identifiers, re-write local profile rows still referenced after the clear, and apply the entire restore as a single transaction so the vault is either the old book or the new one and never a mixture. I also made restore validate the backup's own contents before writing anything, and made export refuse outright if any record fails to decrypt — because exporting a partially readable vault makes the loss permanent.

Result. Cross-device restore works, and is now covered by tests including one that restores into a genuinely second vault. The wider lesson: "the write succeeded" and "the user can see it" are different claims, and only the second one is the feature.

Prepared from the Khazana repository on 30 August 2026. Figures counted from the source tree; performance numbers reproduced from committed benchmark scripts. Sandhosh Sivan · sandhoshsivan00@gmail.com